

B2B SaaS Platform Testing - Multi-Platform Automation

Overview

You're joining the QA team at "WorkFlow Pro" - a B2B project management SaaS platform. The platform serves multiple clients (multi-tenant), works across web browsers and mobile devices, and integrates with various third-party services.

Time Allocation

- **Take-Home Portion:** 90 minutes maximum
 - **Live Discussion:** 30-45 minutes (scheduled separately)
-

Part 1: Debugging Flaky Test Code (30 minutes)

A previous intern wrote this Playwright test for user login functionality. **The test is flaky** - sometimes passes, sometimes fails in CI/CD pipeline.

```
import pytest
from playwright.sync_api import sync_playwright

def test_user_login():
    with sync_playwright() as p:
        browser = p.chromium.launch()
        page = browser.new_page()

        # Navigate to login page
        page.goto("https://app.workflowpro.com/login")

        # Fill login form
        page.fill("#email", "admin@company1.com")
        page.fill("#password", "password123")
        page.click("#login-btn")

        # Verify successful login
        assert page.url == "https://app.workflowpro.com/dashboard"
        assert page.locator(".welcome-message").is_visible()

    browser.close()

def test_multi_tenant_access():
    with sync_playwright() as p:
        browser = p.chromium.launch()
        page = browser.new_page()
```

```
page.goto("https://app.workflowpro.com/login")
page.fill("#email", "user@company2.com")
page.fill("#password", "password123")
page.click("#login-btn")
```

```
# User should only see Company2 data
projects = page.locator(".project-card").all()
for project in projects:
    assert "Company2" in project.text_content()
```

```
browser.close()
```

Your Tasks:

1. **Identify Flakiness Issues:** List all problems that could cause intermittent failures
2. **Explain Root Causes:** Why do these issues happen in CI/CD vs local environment?
3. **Provide Fixes:** Write the corrected version with proper waits, error handling, and reliability improvements

Additional Context (you may need to ask for more):

- App uses dynamic loading for dashboard elements
 - Login process includes 2FA for some users
 - Different tenants have different loading times
 - CI runs on different browsers and screen sizes
-

Part 2: Test Framework Design (25 minutes)

Design a test automation framework structure for this B2B SaaS platform. **Note:** Requirements are intentionally incomplete.

Given Requirements:

- Support web (Chrome, Firefox, Safari) and mobile (iOS, Android) testing
- Handle multiple tenant environments (company1.workflowpro.com, company2.workflowpro.com)
- Test different user roles (Admin, Manager, Employee) with varying permissions
- API testing for backend services
- Integration with BrowserStack for cross-platform testing
- CI/CD pipeline integration

Your Tasks:

1. **Framework Structure:** Design folder structure, base classes, and utilities

2. **Configuration Management:** How to handle multiple environments, browsers, and test data
3. **Identify Missing Requirements:** What questions would you ask about test data management, reporting, parallel execution, etc.?

Format: Use text/pseudocode/diagrams to explain your design

Part 3: API + UI Integration Test (35 minutes)

Create a test that validates the complete flow of project creation through both API and UI layers.

Business Scenario:

1. Create a project via API
2. Verify project appears correctly in web UI
3. Check project is accessible on mobile
4. Validate proper tenant isolation (project only visible to correct company)

API Endpoint Information:

POST /api/v1/projects

Headers: Authorization: Bearer {token}, X-Tenant-ID: {company_id}

Body: {"name": "Test Project", "description": "...", "team_members": [...]}

Response: {"id": 123, "name": "Test Project", "status": "active"}

Your Tasks:

1. **Write Integration Test:** Combine pytest (API) + Playwright (UI) + BrowserStack mobile testing
2. **Handle Test Data:** Show how you'd manage test data across API/UI
3. **Cross-Platform Validation:** Ensure the test works on multiple browsers and mobile devices
4. **Tenant Isolation:** Verify security boundaries between companies

Expected Test Structure:

```
def test_project_creation_flow():  
    # 1. API: Create project  
    # 2. Web UI: Verify project display  
    # 3. Mobile: Check mobile accessibility  
    # 4. Security: Verify tenant isolation  
    pass
```

Your Tasks:

1. **Implement the test** using pytest + Playwright + BrowserStack concepts
2. **Handle edge cases** (network failures, slow loading, mobile responsiveness)
3. **Explain approach** with comments on your testing strategy

Hints: You'll need to make assumptions about authentication, test data cleanup, and mobile testing setup.

Submission Instructions

1. **Create a document** with your responses to all three parts
 2. **Include reasoning** for each technical decision
 3. **List assumptions** about missing requirements or unclear specifications
 4. **Show test framework thinking** - not just individual test cases
 5. **Be prepared** to discuss testing strategy and tool choices in live session
-

Live Session Topics (Preview)

During our video call, we'll discuss:

- Your approach to debugging flaky tests and prevention strategies
 - Framework design decisions and scalability considerations
 - How you'd handle test data management in multi-tenant environments
 - BrowserStack integration strategy and cost optimization
 - CI/CD pipeline integration and parallel execution
 - Monitoring and reporting for test automation
-

Evaluation Criteria

Technical Skills:

- Understanding of test automation tools (Playwright, pytest, BrowserStack)
- Knowledge of web/mobile testing challenges
- API testing and integration concepts
- Framework design and maintainability thinking

Testing Strategy:

- Ability to identify and fix flaky tests
- Understanding of multi-tenant testing challenges
- Cross-platform testing approach
- Test data management strategies

Problem-Solving:

- Systematic debugging approach
- Recognition of missing requirements
- Consideration of real-world constraints (CI/CD, cost, maintenance)

Communication:

- Clear explanation of testing decisions
- Ability to ask clarifying questions about ambiguous requirements
- Understanding of business impact of testing choices

Note: This assessment evaluates your current automation skills and learning potential. We value thoughtful approaches over perfect implementations - focus on demonstrating your testing mindset and technical reasoning.